

SKILL WEEK - 3

Session - 5:

Task -1

To Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases.

CODE :

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200];
    char result[200];
    int i = 0, j = 0;
    int inside = 0;

    printf("Enter text: ");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0') {
        if (input[i] == '\\') {
            inside = !inside;
            i++;
            continue;
        }

        result[j++] = input[i++];
    }

    result[j] = '\\0';

    printf("\\nParsed Text: %s", result);

    if (inside)
        printf("\\nError: Unclosed single quote\\n");
    else
        printf("\\nParsing successful\\n");

    return 0;
}
```

OUTPUT :

```
Compilation terminated.  
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc single_quotes.c -o single_quotes  
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./single_quotes  
Enter text: 'hello world;  
  
Parsed Text: hello world;  
  
Error: Unclosed single quote  
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./single_quotes  
Enter text: 'hello world'  
  
Parsed Text: hello world  
  
Parsing successful  
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./single_quotes  
Enter text: '$HOME'  
  
Parsed Text: $HOME  
  
Parsing successful  
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./single_quotes  
Enter text: 'hello  
  
Parsed Text: hello  
  
Error: Unclosed single quote
```

OBSERVATION :

The program successfully handles single-quoted strings by preserving their literal content and preventing variable expansion. The quotation marks are removed during parsing while the enclosed text remains unchanged. The program also stores the parsed string and validates whether the quotes are properly closed. Edge cases such as an unclosed single quote are detected and reported as parsing errors.

Task-2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

CODE :

```
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

int main() {
    char input[200];
    char result[300];
    int i = 0, j = 0;
    int inside = 0;

    printf("Enter text: ");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0') {
        if (input[i] == '"') {
            inside = !inside;
            i++;
            continue;
        }

        /* Variable expansion */
        if (input[i] == '$' &&
            strncmp(&input[i], "$HOME", 5) == 0) {
            char *home = getenv("HOME");

            if (home != NULL) {
                strcpy(&result[j], home);
                j += strlen(home);
            }

            i += 5;
        }
        else {
            result[j++] = input[i++];
        }
    }

    result[j] = '\0';

    printf("\nParsed Text: %s", result);

    if (inside)
        printf("\nError: Unclosed double quote\n");
    else
        printf("\nParsing successful\n");

    return 0;
}
```

OUTPUT :

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano double_quotes.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc double_quotes.c -o double_quotes
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./double_quotes
Enter text: "hello world"

Parsed Text: hello world

Parsing successful
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./double_quotes
Enter text: "Home directory is $HOME"

Parsed Text: Home directory is /home/ananya

Parsing successful
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./double_quotes
Enter text: "ls -l"

Parsed Text: ls -l

Parsing successful
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./double_quotes
Enter text: "hello world"

Parsed Text: hello world

Error: Unclosed double quote
```

OBSERVATION :

The program successfully handles double-quoted strings while preserving spaces within the quoted content. Unlike single quotes, variable expansion is supported, allowing environment variables such as \$HOME to be replaced with their corresponding values. The program also validates whether double quotes are properly closed and reports errors for invalid quoted commands. Thus, the program demonstrates double-quote parsing, space preservation, variable expansion, and basic validation of quoted input.

Session - 6 :

Task - 1

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

CODE :

```
#include <stdio.h>
#include <string.h>

int main() {
    char input[200], output[200];
    int i = 0, j = 0;

    printf("Enter input: ");
    fgets(input, sizeof(input), stdin);

    while (input[i] != '\0') {
        if (input[i] == '\\' && input[i + 1] != '\0') {
            i++;
            output[j++] = input[i++];
        }
        else {
            output[j++] = input[i++];
        }
    }

    output[j] = '\0';

    printf("Parsed Output: %s", output);

    return 0;
}
```

OUTPUT :

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano escape.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc escape.c -o escape
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./escape
Enter input: hello\ world
Parsed Output: hello world
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./escape
Enter input: hello!\word
Parsed Output: hello!word
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./escape
Enter input: hello\ world\ test
Parsed Output: hello world test
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano escape.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ █
```

OBSERVATION :

The program successfully processes escape sequences by identifying the backslash character and preserving the character that follows it. Escaped spaces and special symbols are handled correctly without losing their original meaning. The program produces the parsed output while preserving the intended characters, demonstrating basic escape-sequence processing and validation with different input cases.

Task - 2

To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

CODE:

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    pid_t pid;

    pid = fork();

    if (pid < 0) {
        printf("Fork failed\n");
        return 1;
    }

    if (pid == 0) {
        printf("Child Process\n");
        printf("PID: %d\n", getpid());

        execlp("ls", "ls", "-l", NULL);

        printf("Execution failed\n");
        return 1;
    }

    else {
        printf("Parent Process\n");
        printf("PID: %d\n", getpid());

        wait(NULL);

        printf("Child process completed\n");
    }

    return 0;
}
```

OUTPUT :

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano escape.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano process.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc process.c -o process
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./process
Parent Process
PID: 1713
Child Process
PID: 1714
total 70
-rwxrwxrwx 1 ananya ananya 16312 Aug 11 13:57 double_quotes
-rwxrwxrwx 1 ananya ananya 954 Aug 11 13:57 double_quotes.c
-rwxrwxrwx 1 ananya ananya 16088 Aug 11 14:06 escape
-rwxrwxrwx 1 ananya ananya 484 Aug 11 14:05 escape.c
-rwxrwxrwx 1 ananya ananya 16168 Aug 11 14:09 process
-rwxrwxrwx 1 ananya ananya 562 Aug 11 14:09 process.c
-rwxrwxrwx 1 ananya ananya 16136 Aug 11 13:53 single_quotes
-rwxrwxrwx 1 ananya ananya 594 Aug 11 13:52 single_quotes.c
Child process completed
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ █
```

```
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ nano process.c
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ gcc process.c -o process
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ ./process
Parent Process
PID: 1750
Child Process
PID: 1751
Execution failed
Child process completed
ananya@Ananya:/mnt/d/OSSP_2520080069/Skill/Week3$ █
```

OBSERVATION :

The program successfully creates a child process using `fork()` and executes an external program using `execlp()`. Arguments are passed to the executed program, while the parent process uses `wait()` to wait for the child to complete. Execution errors are handled when the requested program cannot be launched. Thus, the program demonstrates child-process creation, program execution, argument passing, parent-process management, and command launching.